

90-Day Python DSA Sprint

DAY 01 - Explained Notes: Python Containers, Arrays, and Hashing

What you should understand today

DSA problems are rarely about memorizing code. First decide what information you need to keep while scanning input. Python gives us four important containers for doing that.

Container	Meaning	Use it in DSA when
list []	Ordered and changeable; duplicates allowed.	Order, index, or a collection of answers matters.
tuple ()	Ordered but fixed: it cannot be changed.	A coordinate such as (row, col) needs to be a set/dict key.
set {}	Unique values only; no meaningful order.	You ask: 'Have I seen this value already?'
dict {key: value}	Maps a key to useful information.	You need an index, count, group, or other attached value.

Small syntax examples

`nums = [10, 20, 30]` creates a list. `nums[0]` reads the first item. `nums.append(40)` adds an item. `point = (2, 5)` creates a tuple. `seen = set()` creates an empty set. `score = {'sam': 95}` creates a dictionary.

The key decision

Ask yourself: *What must I remember from earlier elements?* Only whether it appeared: use a set. Its index or count: use a dictionary. This decision is the heart of Day 1.

Big-O: how to describe speed

Big-O tells us how the amount of work grows as the input size n grows. We ignore small constants; $3n + 10$ is still $O(n)$.

Complexity	Meaning	Example
$O(1)$	The work does not grow with n .	<code>nums[0]</code> ; <code>average x in my_set</code>
$O(n)$	One scan through the input.	A single for loop over <code>nums</code> .
$O(n \log n)$	Usually sorting.	<code>sorted(nums)</code>
$O(n^2)$	Every item is compared with many others.	Nested loops for all pairs.

Why a set helps

A list membership check, `x in nums`, may examine every item: $O(n)$. A set membership check, `x in seen`, is average $O(1)$. So a set often converts a repeated-search problem from $O(n^2)$ to $O(n)$.

Walkthrough: first duplicate

Input: `[4, 1, 3, 1, 2]`. Start with `seen = set()`. Read 4: it is not in `seen`, so add it. Read 1: add it. Read 3: add it. Read 1 again: it is already in `seen`, so this is the first duplicate.

```
def contains_duplicate(nums):
    seen = set() # Start with no remembered values
    for num in nums: # Visit one value at a time
        if num in seen: # Have we already seen this exact value?
            return True # Yes - stop immediately
        seen.add(num) # No - remember it for later
    return False # The loop ended with no duplicate
```

For this function: time is $O(n)$ because we scan once. Extra space is $O(n)$ in the worst case because the set can store all numbers.

Dictionary pattern: Two Sum

Question: find two indices whose values add up to target. A slow solution tries every pair. A better solution remembers each earlier value and its index.

```
def two_sum(nums, target):
    seen = {} # number -> index
    for i, num in enumerate(nums):
        needed = target - num # The partner needed to reach target
        if needed in seen: # Was that partner seen earlier?
            return [seen[needed], i]
        seen[num] = i # Remember current number and index
    return []
```

Syntax explained

`enumerate(nums)` gives both the index and the value. For `[3, 2, 4]`, it produces `(0, 3)`, then `(1, 2)`, then `(2, 4)`. `seen[num] = i` means 'use num as the key and save i as its value.'

Trace: nums = [3, 2, 4], target = 6

Current pair	needed	seen before storing	Action
i=0, num=3	3	{}	Store 3:0
i=1, num=2	4	{3: 0}	Store 2:1
i=2, num=4	2	{3: 0, 2: 1}	Found 2 at index 1; return [1, 2]

Important: check needed before storing num. That stops the algorithm from using the same list element twice.

Frequency counting and anagrams

Anagrams have the same letters with the same counts. A dictionary can store a count for each character. For 'aab', the first loop builds {'a': 2, 'b': 1}.

```
def is_anagram(s, t):
    if len(s) != len(t):
        return False # Different lengths cannot be anagrams
    count = {}
    for char in s:
        count[char] = count.get(char, 0) + 1
    for char in t:
        if char not in count:
            return False
        count[char] -= 1
        if count[char] < 0:
            return False
    return True
```

The most useful line

`count.get(char, 0)` means: 'look up char; if it does not exist, pretend its count is 0.' Then add 1. It lets the same line handle both a brand-new letter and a repeated letter.

Group Anagrams

For every word, sort its letters to create the same key for all anagrams. 'eat', 'tea', and 'ate' all become 'aet'. Store words under that key in a dictionary. `' '.join(sorted(word))` means: sort the letters, then join them back into one string.

Product Except Self - the idea only

For each position, multiply the numbers to its left and the numbers to its right. `result = [1] * len(nums)` creates an output list full of 1s. `prefix *= nums[i]` is short for `prefix = prefix * nums[i]`. The reverse loop `range(len(nums)-1, -1, -1)` visits indexes from right to left.

You do not need to memorize advanced patterns on Day 1. Focus first on knowing why a set or dictionary is chosen.

Practice in this order - clickable links

Try the Easy questions first. For each one, first explain the slow solution, then say what a set or dictionary lets you remember.

[1. Two Sum \(Easy\)](#) - Dictionary: value to index.

[2. Contains Duplicate \(Easy\)](#) - Set: already seen?

[3. Valid Anagram \(Easy\)](#) - Dictionary: character frequency.

[4. Group Anagrams \(Medium\)](#) - Dictionary: canonical key to list.

[5. Product of Array Except Self \(Medium\)](#) - Prefix and suffix products.

[6. Longest Consecutive Sequence \(Medium\)](#) - Set: sequence starts and membership.

Day 1 self-check

[] I can say the difference between a set and a dictionary.

[] I can write `seen = set()` and `seen.add(num)`.

[] I understand `count.get(key, 0)`.

[] I can trace Two Sum using a small example.

[] I can state the time and space complexity of the set/dictionary solutions.

Next: Day 02 - two pointers and linked-list foundations.